Section 8: Binary Search

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: DSA Foundations for Interviews

Section 8: Binary Search

- SLIDE 71: Section Intro, Binary Search
- SLIDE 72: Binary Search and the Monotonic Property
- SLIDE 73: Two Templates for Binary Search
- SLIDE 73B: Off-By-One Prevention and the 2-Element Verification Trick
- SLIDE 74: Boundary Finding: First and Last Occurrence
- SLIDE 75: Binary Search on Rotated Sorted Arrays
- SLIDE 76: Binary Search on Answer Space
- SLIDE 76B: Find Peak Element: Binary Search on Unsorted Array
- SLIDE 77: Wrong vs Right: Binary Search Mistakes
- SLIDE 78: How to Present Binary Search in an Interview
- SLIDE 79: Interview Practice: Binary Search
- SLIDE 80: Section Cheat Sheet

C11: DSA Foundations for Interviews

Section 8: Binary Search

SLIDE 71: Section Intro, Binary Search

Section 8: Binary Search

Binary search eliminates half the search space at every step. On a sorted array of one million elements, linear scan checks up to 1,000,000 elements. Binary search checks at most 20. That ratio, O(n) versus O(log n), is the largest single-step complexity improvement you will encounter in interviews.

But binary search is also the technique most likely to produce bugs. Off-by-one errors in loop conditions, boundary updates, and mid-point calculation cause more binary search failures in interviews than wrong algorithmic thinking. This section gives you two templates that eliminate those errors, and a verification technique that catches any remaining mistake before you submit.

Binary search extends far beyond sorted arrays. Any problem where you can define a condition that splits a range into “all satisfy” on one side and “none satisfy” on the other is a binary search problem. This includes searching answer spaces, finding transition points, and locating peaks in unsorted data.

What We’ll Cover:

1. Binary search fundamentals and the monotonic property
2. Two bug-free templates: exact match and boundary search

3. Off-by-one prevention and the 2-element verification trick
4. First and last occurrence in arrays with duplicates
5. Binary search on rotated sorted arrays
6. Binary search on answer spaces (Koko Eating Bananas)
7. Finding peaks in unsorted arrays

Where Binary Search Connects to Other Sections:

Connection	Section	How It Applies
Binary search requires sorted input	Section 9: Sorting	Sort-then-binary-search is a common composite strategy
Two pointers on sorted arrays is an alternative	Section 6: Two Pointers	Two Sum on sorted: two pointers O(n) vs binary search O(n log n)
Sliding window finds optimal subarrays	Section 7: Sliding Window	Some “minimum window” problems can be solved with binary search on answer
Loop-halving pattern from complexity analysis	Section 2: Complexity	O(log n) comes from halving the input at each step

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 72: Binary Search and the Monotonic Property

How Binary Search Works

Start with a range [left, right]. Compute the midpoint. Check a condition at mid. Based on the result, discard the half that cannot contain the answer. Repeat until the range collapses.

Each step eliminates 50% of the remaining search space. After k steps, the remaining space is n / 2^k. The search ends when this reaches 1, so k = log2(n).

Concrete Scale Comparison:

Array Size (n)	Linear Scan (worst case)	Binary Search (worst case)	Speedup
100	100 checks	7 checks	14x
10,000	10,000 checks	14 checks	714x
1,000,000	1,000,000 checks	20 checks	50,000x
1,000,000,000	1,000,000,000 checks	30 checks	33,000,000x

At n = 10^9, the difference between O(n) and O(log n) is the difference between a 10-second timeout and an instant response.

The Monotonic Property: The Real Prerequisite

Binary search does not require a sorted array. It requires a monotonic property: some boundary in the search space where all elements on one side satisfy a condition and all elements on the other side do not.

A sorted array is the most common example. For target = 5 in [1, 2, 3, 4, 5, 6, 7, 8], every element left of 5 is “too small” and every element right of 5 is “too large.” The boundary is clear, so binary search works.

But the monotonic property appears in many other contexts: rotated arrays have a sorted half at every midpoint, answer spaces have feasible and infeasible regions, and peak-finding has an ascending side and a descending side. All are binary searchable.

Classic Binary Search: Find Target in Sorted Array

```java
public int search(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

Trace through [1, 3, 5, 7, 9, 11, 13] searching for 9:

Step	left	right	mid	nums[mid]	Action
1	0	6	3	7	7 < 9, left = 4
2	4	6	5	11	11 > 9, right = 4
3	4	4	4	9	Found at index 4

Three steps to find the answer in a 7-element array. log2(7) rounds up to 3.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 73: Two Templates for Binary Search

The most common source of binary search bugs is mixing pieces from different templates. This section gives you two complete templates. Pick one based on the problem type and use it consistently.

Template 1: Exact Match

Use when searching for a specific value. The loop checks all three cases (equal, less, greater) and returns immediately on match.

```java
public int binarySearchExact(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
```

```
        }
    }
    return -1;
}
```

Loop condition: left <= right (includes single-element range). Both boundaries move past mid: left = mid + 1, right = mid - 1. Returns inside the loop when found, or -1 after exhaustion.

Template 2: Boundary Search

Use when searching for a boundary position: the first element satisfying a condition, the last element below a threshold, or the minimum feasible value in an answer space. The answer is at the boundary itself, and mid might be the answer, so we never skip it.

```
public int binarySearchBoundary(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (condition(nums[mid])) {
            right = mid;        // mid might be the answer, keep it
        } else {
            left = mid + 1;     // mid is definitely not the answer, skip it
        }
    }
    return left;   // left == right, both point to the boundary
}
```

Loop condition: left < right (loop ends when range collapses to one element). The critical asymmetry: right = mid (mid might be answer, keep it in range) versus left = mid + 1 (mid is not the answer, exclude it). The loop terminates with left == right, and that position is the answer.

Template Comparison:

Property	Template 1: Exact Match	Template 2: Boundary
Loop condition	while (left <= right)	while (left < right)
Left update	left = mid + 1	left = mid + 1
Right update	right = mid - 1	right = mid
Return	mid (inside loop) or -1	left (after loop, left == right)
Use when	Find specific value	Find boundary/first position
Risk of infinite loop	No (both sides shrink past mid)	Only if you write left = mid (see 73B)

Interview Tip: State which template you are using before you start coding. “I’ll use the boundary template here because I need the first position where the condition holds, not a specific value.” This tells the interviewer you have a systematic approach to binary search, not an ad-hoc one.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 73B: Off-By-One Prevention and the 2-Element Verification Trick

Why Binary Search Has More Bugs Than Any Other Pattern

Three things can go wrong, and each produces a different failure mode:

Bug Source	What Goes Wrong	Symptom
Wrong loop condition	left <= right with right = mid	Infinite loop on 2-element input
Wrong mid rounding	mid biased left when left = mid	Infinite loop (left never advances)
Wrong boundary update	left = mid instead of left = mid + 1 in Template 2	Infinite loop when left + 1 == right
Mixed templates	Template 1 condition with Template 2 updates	Skips elements or infinite loops

The 2-Element Verification Trick

After writing your binary search code, trace it through a 2-element array [x, y]. If the code terminates correctly for this input, your boundary logic is correct for all inputs.

Why 2 elements? Because this is the smallest input where left != right, and it is the case where off-by-one errors manifest. If left = 0 and right = 1, then mid = 0. If your update sets left = mid (not mid + 1), the range stays [0, 1] forever.

Trace Example: Verify Template 2 with [3, 7], searching for first element >= 5:

Step	left	right	mid	nums[mid]	Condition (>= 5)	Update
1	0	1	0	3	false	left = 0 + 1 = 1
End	left == right == 1					Return 1 (nums[1] = 7 >= 5)

The range collapses correctly. Now verify the opposite case, [3, 7] searching for first element >= 2:

Step	left	right	mid	nums[mid]	Condition (>= 2)	Update
1	0	1	0	3	true	right = 0
End	left == right == 0					Return 0 (nums[0] = 3 >= 2)

Both cases terminate correctly with Template 2. If either case loops infinitely or returns the wrong index, your boundary logic has a bug.

Integer Overflow Prevention

```
// Wrong: overflows when left + right > Integer.MAX_VALUE
int mid = (left + right) / 2;

// Right: always safe
int mid = left + (right - left) / 2;
```

The expression (left + right) can exceed Integer.MAX_VALUE (2,147,483,647) when both values are large. The safe form left + (right - left) / 2 computes the same result without overflow. Always use the safe form. In interviews, writing the overflow-prone version is a minor negative signal. Writing the safe version shows attention to production correctness.

The Cardinal Rule: Never Mix Templates

Template 1 uses left <= right with right = mid - 1. Template 2 uses left < right with right = mid. If you use Template 1's loop condition (<=) with Template 2's right update (= mid), the loop can visit mid repeatedly without shrinking the range. Pick one template and use every component from that template together.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 74: Boundary Finding: First and Last Occurrence

The Problem with Standard Binary Search on Duplicates

Given [1, 2, 2, 2, 3, 4, 5] and target 2, standard binary search (Template 1) finds some occurrence of 2, but which one is unpredictable. If the problem asks for the first or last occurrence, you need to keep searching after finding a match.

Finding the First Occurrence

When nums[mid] == target, do not return. Save the result and continue searching left, because an earlier occurrence might exist.

```
public int findFirst(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            result = mid;
            right = mid - 1;     // keep searching left for earlier occurrence
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return result;
}
```

Finding the Last Occurrence

Same structure, but when nums[mid] == target, save result and continue searching right.

```
public int findLast(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            result = mid;
            left = mid + 1;      // keep searching right for later occurrence
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return result;
}
```

Trace through [1, 2, 2, 2, 3, 4, 5] with target = 2:

Finding first occurrence:

Step	left	right	mid	nums[mid]	Action	result
1	0	6	3	2	Match, save, search left	3
2	0	2	1	2	Match, save, search left	1
3	0	0	0	1	Too small, search right	1
4	1	0			left > right, exit	1

First occurrence at index 1.

Finding last occurrence:

Step	left	right	mid	nums[mid]	Action	result
1	0	6	3	2	Match, save, search right	3
2	4	6	5	4	Too large, search left	3
3	4	4	4	3	Too large, search left	3
4	4	3			left > right, exit	3

Last occurrence at index 3.

The Structural Difference

Both solutions are identical except for a single line. On match: right = mid - 1 (first occurrence, search left) versus left = mid + 1 (last occurrence, search right). This one-line difference is a common interview question: “How would you modify your solution to find the last occurrence instead?”

Practical Application: Count of Target

Once you can find first and last occurrence, the count of a target element is lastIndex - firstIndex + 1. This is an O(log n) alternative to the O(n) linear scan for counting in a sorted array.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

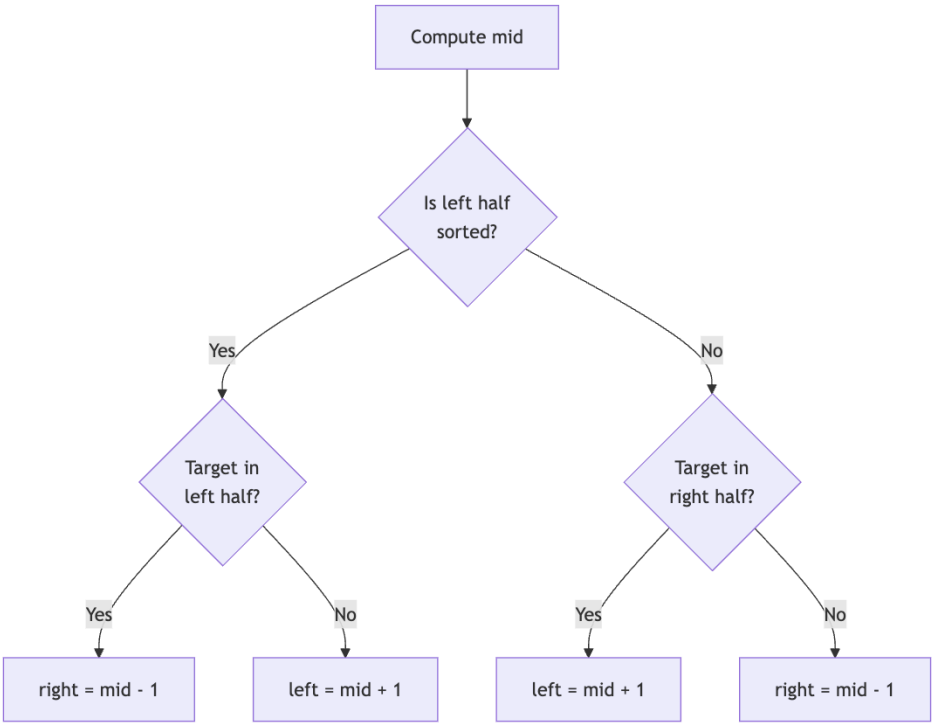
SLIDE 75: Binary Search on Rotated Sorted Arrays

The Setup

A sorted array [0, 1, 2, 3, 4, 5, 6, 7] rotated at the pivot becomes [4, 5, 6, 7, 0, 1, 2]. The array is no longer globally sorted, but it has a critical property: at any midpoint, one half is always fully sorted.

The Key Insight

When you compute mid, either the left half [left..mid] is sorted or the right half [mid..right] is sorted. Exactly one of these is always true. Determine which half is sorted, check if the target falls within that sorted range, and eliminate the other half.



Full Java Solution:

```java
public int search(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        }

        if (nums[left] <= nums[mid]) {
            if (target >= nums[left] && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            if (target > nums[mid] && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
```

```
    return -1;
}
```

Trace through [4, 5, 6, 7, 0, 1, 2] searching for 0:

Step	left	right	mid	nums[mid]	Sorted Half	Target in Sorted Half?	Action
1	0	6	3	7	Left [4,5,6,7] sorted	0 not in [4,7]	left = 4
2	4	6	5	1	Right [1,2] sorted	0 not in [1,2]	right = 4
3	4	4	4	0	Found		Return 4

Three steps. O(log n) time, O(1) space. The key decision at each step: identify the sorted half, check if the target falls in it, and eliminate the appropriate side.

Why nums[left] <= nums[mid] uses <= (not <)

When left == mid (which happens when there are only 2 elements left), nums[left] == nums[mid]. The left half is a single element and is trivially sorted. The <= ensures this case is handled correctly. Using strict < would misidentify which half is sorted.

Follow-up: What About Duplicates?

If the array can contain duplicates, like [2, 5, 6, 0, 0, 1, 2], then when nums[left] == nums[mid] == nums[right], you cannot determine which half is sorted. The workaround: increment left and decrement right to shrink the ambiguous range.

```
if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
    left++;
    right--;
}
```

This degrades the worst case to O(n) when all elements are equal, but the average case remains O(log n). Know this follow-up exists, but the primary interview problem uses distinct elements.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 76: Binary Search on Answer Space

The Conceptual Shift

In classic binary search, you search through the data for a specific value. In answer-space binary search, you search through the range of possible answers for the optimal one.

The setup: the problem asks “find the minimum (or maximum) X such that some condition is satisfied.” The condition is monotonic: if X works, every X larger (or smaller depending on the problem) also works. This monotonic property means binary search applies, even though you are not searching an array at all.

Worked Example: Koko Eating Bananas (LeetCode 875)

Problem: Koko has n piles of bananas. She can eat at most speed bananas per hour from a single pile. She has h hours to eat all the bananas. Find the minimum speed that lets her finish in time.

Step 1: Understand

Piles: [3, 6, 7, 11], hours h = 8. Speed must be an integer. At speed 4, pile of 11 takes ceil(11/4) = 3 hours. Total hours = ceil(3/4) + ceil(6/4) + ceil(7/4) + ceil(11/4) = 1 + 2 + 2 + 3 = 8. Exactly fits.

Step 2: Identify the Pattern

"Find minimum speed such that total hours <= h." The condition is monotonic: if speed k finishes in time, any speed above k also finishes in time. Binary search on the answer space [1, max(piles)].

Step 3: Plan

Search space: speed ranges from 1 (minimum possible) to max(piles) (eats any pile in one hour). For each candidate speed, compute total hours needed. If total <= h, try slower. If total > h, need faster. Template 2 boundary search.

Step 4: Implement

```
public int minEatingSpeed(int[] piles, int h) {
    int left = 1;
    int right = 0;
    for (int pile : piles) {
        right = Math.max(right, pile);
    }

    while (left < right) {
        int mid = left + (right - left) / 2;
        int hoursNeeded = 0;

        for (int pile : piles) {
            hoursNeeded += (pile + mid - 1) / mid;   // ceiling division
        }

        if (hoursNeeded <= h) {
            right = mid;        // speed mid works, try slower
        } else {
            left = mid + 1;     // speed mid too slow, need faster
        }
    }
    return left;
}
```

The expression (pile + mid - 1) / mid computes the ceiling of pile / mid using integer arithmetic, avoiding floating-point division.

Step 5: Verify

Trace through piles = [3, 6, 7, 11], h = 8:

Step	left	right	mid (speed)	Hours at mid	Fits in 8h?	Action
1	1	11	6	1+1+2+2 = 6	Yes	right = 6
2	1	6	3	1+2+3+4 = 10	No	left = 4
3	4	6	5	1+2+2+3 = 8	Yes	right = 5
4	4	5	4	1+2+2+3 = 8	Yes	right = 4
End	left == right == 4					Return 4

Minimum eating speed: 4 bananas per hour. Complexity: O(n * log(max(piles))). The binary search runs log(max(piles)) iterations, and each iteration scans all n piles.

How to Recognize Answer-Space Problems:

Signal in Problem Statement	What to Search	Condition
"Minimum speed/capacity/size such that..."	[1, max possible]	canFinish(mid) is monotonic
"Maximum distance/value such that..."	[1, max possible]	isValid(mid) is monotonic
"Split array into k parts minimizing max sum"	[max element, total sum]	canSplit(mid, k) is monotonic

Signal in Problem Statement	What to Search	Condition
“Minimum days to make m bouquets”	[1, max(bloomDay)]	canMake(mid) is monotonic

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 76B: Find Peak Element: Binary Search on Unsorted Array

Breaking the “Binary Search Equals Sorted” Misconception

Find Peak Element (LeetCode 162): an array where no two adjacent elements are equal. A peak is an element greater than both its neighbors. Return the index of any peak. The array is not sorted.

Binary search still works because the monotonic property exists, even without sorted order.

The Monotonic Property for Peaks

If nums[mid] < nums[mid + 1], then the right side is ascending at mid. A peak must exist to the right (the array cannot ascend forever because the boundary is defined as nums[-1] = nums[n] = negative infinity). If nums[mid] >= nums[mid + 1], then a peak exists at mid or to its left. This is a monotonic condition: one side always contains a peak.

```java
public int findPeakElement(int[] nums) {
    int left = 0;
    int right = nums.length - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] < nums[mid + 1]) {
            left = mid + 1;     // peak is to the right
        } else {
            right = mid;        // peak is at mid or to the left
        }
    }
    return left;
}
```

Trace through [1, 2, 1, 3, 5, 6, 4]:

Step	left	right	mid	nums[mid]	nums[mid+1]	Comparison	Action
1	0	6	3	3	5	3 < 5	left = 4
2	4	6	5	6	4	6 > 4	right = 5
3	4	5	4	5	6	5 < 6	left = 5
End	left == right == 5						Return 5

Index 5 has value 6, which is greater than both neighbors (5 and 4). Correct peak found.

Why This Matters for Interview Understanding

This problem proves that binary search is about the monotonic property, not about sorted order. When an interviewer gives you an unsorted array and you say “I’ll use binary search,” that demonstrates a deeper understanding of the technique than candidates who only associate binary search with sorted data.

Time O(log n). Space O(1). On an unsorted array.

SLIDE 77: Wrong vs Right: Binary Search Mistakes

Mistake 1: Using left = mid in Template 2

Wrong:

```
while (left < right) {
    int mid = left + (right - left) / 2;
    if (condition(nums[mid])) {
        right = mid;
    } else {
        left = mid;     //BUG: infinite loop when left + 1 == right
    }
}
```

When left = 0 and right = 1, mid = 0, and left = mid keeps left at 0. The range never shrinks. The loop runs forever.

Right:

```
left = mid + 1;     // mid is not the answer, exclude it
```

Mistake 2: Assuming Binary Search Only Works on Sorted Arrays

Wrong: “The array isn’t sorted, so binary search doesn’t apply.”

Right: Binary search works on any search space with a monotonic property. Find Peak Element works on unsorted arrays. Koko Eating Bananas searches an answer range, not an array. The question is whether the search space can be halved by a condition, not whether the data is sorted.

Mistake 3: Integer Overflow in Mid Calculation

Wrong:

```
int mid = (left + right) / 2;    // overflows when left + right > 2^31 - 1
```

Right:

```
int mid = left + (right - left) / 2;    // always safe
```

Mistake 4: Mixing Template Components

Wrong: using while (left <= right) with right = mid. This combination means that when left == right, the loop still runs, and right = mid does not shrink the range. Infinite loop.

Right: pick one template. Template 1: while (left <= right) with right = mid - 1. Template 2: while (left < right) with right = mid. Never cross the templates.

Mistake 5: Forgetting to Handle the Result After Template 2

Wrong:

```
while (left < right) {
    // ... boundary search
}
return left;    // but not checking if nums[left] actually satisfies the condition
```

Template 2 finds the boundary position, but if no element satisfies the condition (e.g., target is larger than all elements), left lands at the last index. You may need to verify that nums[left] actually meets the requirement before returning.

Right: After Template 2 returns, check: does nums[left] satisfy the condition? If not, the answer does not exist.

Summary of Common Binary Search Bugs:

Bug	Cause	Fix
Infinite loop	left = mid without +1 in Template 2	Always left = mid + 1 when mid is excluded
Infinite loop	Template 1 condition with Template 2 updates	Never mix templates
Wrong answer	Not verifying result after boundary search	Check nums[left] after loop
Runtime crash	(left + right) / 2 overflow	Use left + (right - left) / 2
Wrong answer	Using strict < instead of <= in sorted half check	Trace through 2-element input to verify

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 78: How to Present Binary Search in an Interview

Introducing Standard Binary Search

"The array is sorted, so I can use binary search to find the target in O(log n) time instead of scanning linearly in O(n). I'll maintain a range [left, right] and halve it at each step based on comparison with the midpoint."

Introducing Boundary Search

"I need the first position where the condition holds, not a specific value. I'll use binary search with the boundary template: when the condition is true at mid, I narrow from the right (right = mid, since mid might be the answer). When false, I narrow from the left (left = mid + 1, since mid is definitely not the answer). The loop ends when left equals right, which is the boundary."

Introducing Answer-Space Search

"The answer must be between 1 and max(piles). I can binary search this range because the feasibility is monotonic: if speed k finishes in time, any speed above k also finishes in time. For each candidate speed, I compute the total hours needed and compare to the limit."

Handling Follow-up Questions:

Interviewer Says	What They Mean	How to Respond
"What if there are duplicates?"	First/last occurrence or rotated with dupes	Explain result-variable approach or left++/right– workaround
"Can you do this in O(log n)?"	Current solution is O(n) and binary search exists	Look for monotonic property in the search space
"What if the array is rotated?"	Standard binary search won't work directly	Identify sorted half at each midpoint
"Walk me through a small example"	They want to see your code is correct	Use the 2-element verification trick

The Strongest Interview Move with Binary Search

When you identify that a problem has an answer-space binary search solution, say: "The answer lies in the range [X, Y]. The feasibility condition is monotonic, so I can binary search the answer space." This framing shows the interviewer you understand binary search as a general technique, not just as array lookup. Most candidates never discover answer-space binary search during an interview. Recognizing it is a strong differentiator.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 79: Interview Practice: Binary Search

Practice Prompt: "Search for a target value in a rotated sorted array. The array was originally sorted in ascending order and then rotated at an unknown pivot."

How to Walk Through This in an Interview:

Step 1: Understand (30 seconds)

"The array was sorted and then rotated, so it looks like [4, 5, 6, 7, 0, 1, 2]. No duplicates. I need to find the index of a target value, or return -1 if it's not present. The expected time complexity is O(log n), which tells me binary search."

Step 2: Identify Pattern (30 seconds)

"This is a modified binary search. The key insight is that at any midpoint, one half of the array is guaranteed to be sorted. I can determine which half is sorted by comparing nums[left] with nums[mid], then check if the target falls within the sorted half."

Step 3: Plan (45 seconds)

"I'll use Template 1 since I'm looking for an exact match. At each step: check if mid is the target. If not, determine which half is sorted. If the target falls in the sorted range, search that half. Otherwise, search the other half."

Step 4: Implement

```
public int search(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        }

        if (nums[left] <= nums[mid]) {
            if (target >= nums[left] && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            if (target > nums[mid] && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
    return -1;
}
```

Step 5: Verify

"Let me trace [4, 5, 6, 7, 0, 1, 2] searching for 1. Mid starts at index 3, value 7. Left half [4,5,6,7] is sorted. Target 1 is not in [4,7], so search right. Next mid is index 5, value 1. Found. Return 5."

"Edge cases: single element array, target at rotation point, target not present. Time O(log n), space O(1)."

Common Fresher Mistakes on Rotated Array Search:

Mistake	Why It Fails	Fix
Using standard binary search directly	Sorted order is broken at the rotation point	Check which half is sorted at each step
Strict < instead of <= for sorted half check	Mishandles the case when left == mid	Use nums[left] <= nums[mid]
Not handling target at the endpoints	Off-by-one in range checks	Use >= and <= in the range comparisons
Trying to find the pivot first, then binary search	Works but is two passes and harder to get right	Single-pass approach is cleaner

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 80: Section Cheat Sheet

Template Quick Reference:

Template	Loop	Left Update	Right Update	Returns	Use When
Template 1: Exact Match	while (left <= right)	left = mid + 1	right = mid - 1	mid (found) or -1	Find specific value
Template 2: Boundary	while (left < right)	left = mid + 1	right = mid	left (== right, boundary)	First/last position, min feasible

When to Use Which Variant:

Problem Type	Template	Key Logic
Find target in sorted array	Template 1	Standard three-way comparison
First occurrence of target	Template 1 + result variable	On match: save result, search left
Last occurrence of target	Template 1 + result variable	On match: save result, search right
Search in rotated sorted array	Template 1	Identify sorted half, check target range
Minimum X such that condition holds	Template 2	Condition true: right = mid. False: left = mid + 1
Find peak element	Template 2	Compare mid with mid+1 for direction
Count of target in sorted array	First + Last occurrence	count = last - first + 1

Pattern Recognition Signals:

Signal in Problem	First Pattern to Try	Complexity
“sorted array” + find value	Standard binary search	O(log n)
“sorted array” + “first/last position”	Boundary search with result variable	O(log n)
“rotated sorted array”	Modified binary search, identify sorted half	O(log n)
“minimum X such that condition”	Binary search on answer space	O(n * log(range))
“peak” or “transition point”	Boundary search on unsorted data	O(log n)
“sorted array” + find pair	Two pointers (Section 6) often better	O(n)

Off-By-One Prevention Checklist:

1. Always use mid = left + (right - left) / 2 (overflow safe)
2. Never use left = mid in Template 2 (infinite loop)
3. Trace through a 2-element array [x, y] before submitting
4. Never mix Template 1 loop condition with Template 2 boundary updates

5. After Template 2, verify that nums[left] actually satisfies the condition

Key Numbers:

n	log2(n)	Binary Search Steps
100	~7	7
10,000	~14	14
1,000,000	~20	20
10^9	~30	30

How This Connects to Other Sections:

This Section's Concept	Connects To	How
Binary search requires sorted input	Section 9: Sorting	Sort first, then binary search is a common strategy
O(log n) from halving search space	Section 2: Complexity	Same pattern as loop that halves input
Finding pairs in sorted arrays	Section 6: Two Pointers	Two pointers O(n) is often preferred over binary search O(n log n) for pair problems
Answer-space search on window sizes	Section 7: Sliding Window	Some "minimum window" problems can be binary searched
Merge Sort uses divide-and-conquer halving	Section 9: Sorting	Same O(log n) depth from splitting in half
Binary search trees (BST lookup)	C6: Trees	BST search is binary search applied to tree structure

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina